

Cyber Verification Program 申請パッケージ

ai-agent-attack-defense — 防御研究・統制証拠一式 / 2026-10-10 更新

プロジェクト: ai-agent-attack-defense (w-index-m)

目的: Gambit Security レポート (2026-09-22) の AI エージェント攻撃 (Strix / Cairn / Hermes) を題材にした 防御研究と小ツール群。本パッケージは Anthropic [Cyber Verification Program](#) (公式ページ公開日 2026-10-06) の申請時に提示する「統制・責任ある取り扱い・技術的検証能力」の証拠を 1 冊に束ねたもの。

申請サマリ

- 想定ティア:
 - 防御研究 → **Defense Access** (個人は公式条件の「報告済み脆弱性の実績」を別途示す)
 - 組織として認可ラボでの攻撃的テスト → **Red Team Access** (組織のみ)
- 責任ある取り扱い: 対象は所有/認可物に限定 (README・SECURITY.md・CONTRIBUTING.md に明記、MIT)。netcheck は範囲外を実行前拒否・同意必須・実行ログ。攻撃ツールの解析は静的レビューが中心で、実行したのは攻撃を伴わない検証 (Strix の LLM 非依存 PoC、Cairn の mock) のみ。攻撃レシピや安全機構の除去は作成・再構成しない方針を文書化。
- 隔離・運用統制: 認可ラボ・ランブック、隔離ネットワーク設計、攻撃なし mock 検証 (実機確認済み)。
- 技術的検証能力: Strix のローカル実行 PoC、検知ルール (R1-R7)、定義の一元化と CI の整合性チェック。
- データ保持: モニタリングのためのデータ保持が必須であることを理解している。

収録ドキュメント

- CVP 適用メモ&ティア対応表 (cvp-readiness.md) — 公式の記載 (2026-10-06 公開) に合わせて更新
- 3ツール横断 防御・検知ノート (defense-detection-notes.md) — 検知ルール R1-R7、MITRE ATT&CK 対応、トリアージ手順を含む
- Cairn 認可ラボ・ランブック (cairn-lab/cairn-authorized-lab-runbook.md)
- Strix ローカル実行バックエンド設計・PoC (strix-local-backend-notes.md)

付随リソース (本冊子には全文は載せず、リポジトリに収録):

- security-scan/detections/ — R1-R7 の実スタック向けルール (Sigma / Falco / Datadog / Elastic)。
- security-scan/artex-analysis.* — ARTEX の防御解析 (静的レビューのみ)。
- security-scan/zgrab2-notes.md — スキャナ出力の取り込みと、対象を許可範囲に絞るガード (scripts/zgrab2_guard.py)。

本パッケージは防御・統制の観点に限定し、攻撃の再現手順や安全機構の回避方法は含まない。

Cyber Verification Program (CVP) 適用メモ & 申請準備

English: [cvp-readiness.en.md](#)

Anthropic の [Cyber Verification Program](#) (CVP = サイバー検証プログラム) に、本プロジェクト (ai-agent-attack-defense: Gambit レポートの AI エージェント攻撃 Strix/Cairn/Hermes を題材にした**防御研究+小ツール**) をどう対応づけ、どのティアを申請するかを整理したもの。

出典と鮮度: 公式ページ (公開日 2026-10-06) を **2026-10-10** に確認して反映した。制度は更新されうるので、申請前に必ず公式ページで再確認すること。公式に書かれていない点は「記載なし」と明記し、推測で埋めない。

1. CVP 3 ティア (公式の記載)

ティア	対象	許可範囲	申請主体	審査
Defense Access	防御 (SOC・インシデント対応・マルウェアのリバースエンジニアリング・脆弱性の分析/検証)	防御業務向けに、ブロックの分類器を緩める	組織 (企業・NPO・大学・政府の防御チーム、重要インフラ事業者、小規模セキュリティ企業、OSS メンテナ) と、「 報告済み脆弱性の実績 」のある個人研究者	数日为目标
Red Team Access	認可されたペンテスト・レッドチーム (Defense の全てを含む)	組織が認可を受けた対象に限る 。物理的被害・大規模な妨害につながる行為 (ランサムウェアの展開、物理システムの破壊、 高リスクな安全システムへのペンテスト など) は引き続きブロック	組織のみ (社内レッドチーム、政府レッドチーム、ペンテスト会社)。 個人は不可	数週間。審査中は Defense Access に登録される
Specialized Access	人命や市場に影響しうる安全重要システム (航空機の OS、電力網、通信網、銀行間送金、政府の行政網など) をテストする認可を受けた、限られた検証済み組織	サイバー分野のブロックが最も少ない	組織 (全組織を 米政府と連携して詳細に審査)	記載なし

共通事項

- モデル:** 全ティアで Claude Opus 5.5 / Sonnet 5.5 / Mythos 5.1 と今後のモデルを利用可能。

- ・ 一般提供モデルでもできること：一般提供モデル（Opus 5.5 / Fable 5.1 / Sonnet 5.5）は、サイバー作業の多くを保守的にブロックする。ただし、コードレビュー、既知の問題のパッチ適用、自分のソースコードの脆弱性発見、セキュリティアラートのトリアージには引き続き使える。→ これらは CVP なしでも可能。
- ・ 申請：ポータル <https://portal.anthropic.com/programs/cvp> 。全申請者が検証され、ティアに必要なセキュリティ統制の証明を求められる（具体的な書類は公式に記載なし）。
- ・ データ保持：登録組織は、サイバー悪用の監視のためデータ保持が必須。Enterprise Frontier Safeguards（EFS：ゼロ保持＋防護、データを利用者管理のクラウドに置ける）は「今秋後半」予定。それまでは、Fable 5.1 / Mythos 5.1 のゼロ保持アクセスを持つ組織は、CVP もゼロ保持で利用可。EFS の関心登録：<https://claude.com/form/enterprise-frontier-safeguards>
- ・ 提供基盤：Claude Platform / Google Cloud Vertex AI / Microsoft Foundry。Amazon Bedrock は EFS の対象顧客のみ。
- ・ 既存メンバー：既存の CVP メンバーは従来のモデルの設定を維持し、新モデルは自動で評価される。Project Glasswing の既存メンバーは Specialized Access に移行（現行モデルは再承認不要）。
- ・ ワークスペース：管理者が、アクセスを割り当てるワークスペースを指定する（手順：[Claude Console のサポート記事](#)）。
- ・ 誤ブロック／却下の異議申立て：<https://claude.com/form/cyber-block-false-positive-report-cvp-rejection-appeal>

2. 本プロジェクトの活動 → ティア対応表

活動（本リポジトリ）	性質	対応ティア
<code>vuln_triage.html</code> / <code>scripts/summarize.py</code> （スキャン結果の集計・トリアージ。SARIF / netcheck / zgrab2 の取り込みを含む）	防御（脆弱性の分析・検証）	Defense
<code>netcheck/</code> （所有資産の確認専用チェック、確認だけ・攻撃なし）、 <code>scripts/zgrab2_guard.py</code> （スキャナに渡す対象を許可範囲に限定）	防御	Defense
<code>security-scan/detections/</code> （Sigma / Falco / Datadog / Elastic の検知ルール R1-R7）	防御（検知の設計）	Defense
<code>.github/workflows/security-scan.yml</code> （Semgrep/Trivy/gitleaks CI）と、定義のズレを止める整合性チェック	防御（コード/依存の検証）	Defense
Strix / Cairn / Hermes / ARTEX のコード・アーキテクチャの静的解析（ <code>*-notes.md</code> , <code>artex-analysis.*</code> ）	防御（攻撃ツールの解析）	Defense
Strix のローカル実行 PoC（ <code>strix-local-poc/</code> 、良性コードへの静的解析）	防御寄り／研究	Defense
Cairn を認可ラボで実際に走らせる（ <code>cairn-lab/</code> ランプック）	攻撃的テスト	Red Team（組織のみ）
安全重要システム（電力/通信等）への実テスト	安全重要	Specialized（該当時のみ・米政府と連携）

読み方：現状の成果物の大半は Defense の射程。攻撃ツールを認可ターゲットへ能動的に走らせる段になって初めて Red Team（組織申請）が必要。

3. 申請準備：本リポジトリが示せるもの／示せないもの

CVP は「必要なセキュリティ統制の証明」を求める（具体書類は記載なし）。現状の資産で示せるもの：

- **責任ある取り扱い**
 - README・SECURITY.md・CONTRIBUTING.md に「自分が所有・認可した対象にのみ使用」「防御・検知の設計に限定」を明記。LICENSE は MIT。
 - netcheck は許可レンジ外を**実行前に拒否**／同意必須／ローカル限定トークン／実行ログ。
scripts/zgrab2_guard.py も同じ判定を再利用し、範囲外が 1 つでも混ざれば何も出力しない。
 - 攻撃ツールの解析は、Hermes・ARTEX・zgrab2 は**静的レビューのみ（実行・ビルドしない）**。実行したのは攻撃を伴わない検証だけ（Strix は LLM を使わない PoC を良性コードに対して、Cairn は攻撃なしの mock エンジン）で、**実対象への攻撃は行っていない**。攻撃レシピや安全フィルタ除去の再構成は扱わない方針を文書化（各解析に明記）。
- **隔離・運用統制の設計**
 - cairn-lab/cairn-authorized-lab-runbook.md：実行前チェックリスト、ネットワーク隔離、egress 制限、使い捨て/スナップショット、監視、禁止事項。
 - cairn-lab/dispatch_mock_local.yaml：**攻撃せずにエンジンのみ検証**した構成。
- **技術的検証能力**
 - strix-local-poc/（再現手順・パッチ・検証ログ）、detections/（R1-R7）、shared/taxonomy.json + CI の整合性チェック。
- **秘密情報管理**：.gitleaks.toml / .pre-commit-config.yaml。CI は緑。

示せない／足りないもの（正直に）

- 個人で Defense を申請する条件は「報告済み脆弱性の実績」。本リポジトリは防御研究とツールであり、脆弱性報告の実績の代わりにはならない。
- 組織としての本人確認・組織の統制（Red Team の場合）。
- データ保持とモニタリングを受け入れられるか（機密・NDA 案件を扱う場合は要検討）。

秘密情報（LLM キー等）はコミットしない運用を継続する。

4. 推奨アクション

1. まず「CVP なしで足りるか」を確認する。コードレビュー、スキャン結果やアラートのトリアージ、自分のコードの脆弱性発見、防御ルールの作成は、公式上も一般提供モデルで可能。現状の作業の多くはこれに当たるので、**今すぐ必須ではない**。
2. 個人で Defense Access を目指す場合：条件は「報告済み脆弱性の実績」。実績がまだ無いなら、（a）自分の資産／認可範囲／バグバウンティの範囲で、正規の脆弱性報告を行って実績をつくる、（b）所属組織を通じて申請する、を検討する。本リポジトリは、**統制・責任ある取り扱い・技術力を示す補強材料**として添える（審査は数日を目標）。申請フォームの回答案：[cvp-application-form-draft.md](#)。

3. 組織として認可ラボ／本番ペネテストまで行う場合 → Red Team Access（組織申請・数週間、審査中は Defense に登録）。前提：対象の所有/書面認可、隔離ラボ、データ保持要件の充足。§3 の隔離設計・ランブック・mock 検証を「統制の証明」として提出。
4. データ保持の扱いを先に決める：モニタリングのためのデータ保持が前提。ゼロ保持が必要ななら、Fable 5.1 / Mythos 5.1 のゼロ保持アクセス、または EFS（今秋後半予定）を確認する。
5. 不変の一線：Red Team でも、物理的被害・大規模な妨害につながる行為（ランサム展開、物理システムの破壊、高リスクな安全系へのペネテスト等）はブロック対象。本プロジェクトの方針（攻撃レシピ・安全フィルタ除去を作らない）と整合。

参考

- Anthropic — Cyber Verification Program（公開日 2026-10-06）
 - Project Glasswing と従来の CVP を統合した拡張枠組み。「サイバーセキュリティは本質的に二面性がある」ため、検証された防御者に限定して提供。
 - Glasswing の成果：パートナーが少なくとも 12.9 万件の検証済みソフトウェア脆弱性を報告（2026/4-7）。Anthropic 自身の OSS スキャンでさらに 5,500 件（2026/4-10）。うち 3.3 万件超が critical/high。公式は過小計上で、実際の影響は「少なくとも 5 倍」と推定。
 - CyScenarioBench (Opus 5.5、10 シナリオ × 5 回 = 各ティア 50 試行)：CVP なしは最初のプロンプトで全てブロック。Defense は 50 件中 46 件がどこかでブロック（4 件は成功）。Red Team はブロックなしで 50 件中 34 件を完遂（防護なしと「実質同等」）。→ Defense は悪性の多段攻撃をほぼ止める設計。
- 申請：<https://portal.anthropic.com/programs/cvp> / ティア詳細の Help Center:
<https://support.claude.com/en/articles/14604842-real-time-cyber-safeguards-on-claude-opus-and-sonnet>

3ツール横断 防御・検知ノート (Strix / Cairn / Hermes)

Gambit レポートの攻撃連鎖（偵察→侵入→統括/事後）を、**防御側の観測点と対策**に振って整理したもの。攻撃手順のレシピではなく、「何が観測され、どこで止められるか」に焦点を当てる。

検知/防御タグ R1-R7 / ③ とガバナンス G1-G6 の正式な定義は `../shared/taxonomy.json`（唯一の定義）にあり、各ツールの追従は CI の `tools/check_consistency.py` が検査する。

出典: Gambit Security / 各ツール: Strix / Cairn / Hermes

攻撃連鎖の役割分担（本セッションの解析より）

段階	ツール	やること	技術的特徴
偵察・スキャン	Strix	攻撃面の発見・脆弱性探索	Docker サンドボックスで実行、caido プロキシで HTTP 捕捉、nmap/ffuf/playwright 等を同梱
探索的侵入	Cairn	目標 (shell/flag) まで自走探索	黒板サーバ(:8000)+fact-intent グラフ、00DA ワーカーを per-project コンテナで起動 (NET_RAW/NET_ADMIN)
統括・事後	Hermes	キャンペーン統括・事後作業	汎用エージェント、多数のスキル、Telegram/Discord/Slack ゲートウェイ、cron、サブエージェント、7 実行バックエンド

共通の土台: LLM API への継続的な egress（レポートでは OpenRouter 経由、Claude/DeepSeek 等）。これが 3 ツール共通の最も安定した観測点。

段階別：観測できる兆候 → 防御/検知

A. 偵察・スキャン (Strix 的挙動)

観測点:

- 単一ソースからの広域ポート/パス走査、短時間の多数リクエスト、既知スキャナの UA/挙動。
- 送信元が 自動化ブラウザ (playwright/CDP) 由来のフィンガープリント。
- 外向きに LLM API への定期的・バースト的 HTTPS (スキャン判断のたびに推論)。

防御/検知:

- WAF/レート制限、スキャン検知 (多数 404/403・パス列挙)、ボット対策。

- 管理画面・機微エンドポイントの認証必須化+IP 制限/VPN 内側（`netcheck` が「管理画面 URL の応答」を確認対象にしているのと表裏）。
- セキュリティヘッダ整備（CSP/HSTS/XFO/nosniff）。決済ページの CSP は `skimmer` 注入対策の要。

B. 探索的侵入（Cairn 的挙動）

観測点：

- 内部に 黒板サーバ(:8000 / FastAPI) と、`per-project` ワーカーコンテナが多数起動（イメージ名 `cairn-worker-container`、`NET_RAW/NET_ADMIN` 付与）。
- ワーカーからの反復的な試行→観測→再試行パターン（00DA。失敗しても別 intent で粘る）。
- 継続的な LLM API egress と、対象への断続的なプローブ。

防御/検知：

- EDR/コンテナランタイム監視：`NET_RAW/NET_ADMIN` 付きコンテナの新規起動、未知イメージの pull、`host` ネットワークモードのコンテナを警戒。
- 内部セグメンテーション（lateral movement 阻止）、重要資産への最小権限・多要素。
- 送信ドメイン allowlist（LLM API への常時 egress は正規業務でなければ異常）。

C. 統括・事後（Hermes 的挙動）

観測点：

- メッセージングアプリ（Telegram/Discord/Slack/Signal）への外向き接続＝C2 的チャネルになり得る。
- `cron`/スケジューラによる永続化、サブエージェントの大量スポン、スキルの動的生成/自己改変。
- 「自身の安全フィルタを外す」類のスキル追加（Gambit が 78/121 の攻撃スキル、うちフィルタ除去を指摘）。

防御/検知：

- 外向きメッセージング API への接続監視（サーバ/本番ホストからの Telegram 等は強い異常シグナル）。
- 永続化監視（新規 `cron`/サービス/スタートアップ項目）、プロセス系統の異常（自動化された連続ツール実行）。
- LLM 利用ガバナンス：安全機構の除去・迂回を図る挙動は方針・契約・CVP の不変の一線（物理被害/大規模妨害/ランサム/安全系無効化はブロック）に照らして遮断・通報。

横断的な「効く」対策（優先順）

1. 外向き通信の allowlist — LLM API・メッセージングへの常時 egress は最強の共通シグナル。本番/サーバ系からの当該通信を既定で遮断・監視。
2. コンテナ/実行環境の可視化 — 特権 cap 付きコンテナ、未知イメージ、`host` ネットの起動を検知。
3. 攻撃面の最小化 — 管理画面の隔離、DB/サービスをネット直結にしない（`netcheck` の RISKY ポート群が示す露出を塞ぐ）。

- 4. 決済・機微ページの CSP — skimmer/カード窃取コード注入の防止。
- 5. 永続化・自動化の監視 — cron/スケジュール、連続自動ツール実行、サブエージェント大量生成。
- 6. ログと相関 — スキャン痕跡+LLM egress+特権コンテナ+外向きメッセージングを時系列で相関させると、単発では見えない「自律エージェント攻撃」が像を結ぶ。

本リポジトリ資産との対応

- netcheck/ : A の露出面 (RISKY ポート、ヘッダ、管理画面応答) を自分の資産で確認するための道具。
- vuln_triage.html / scripts/summarize.py : スキャナ結果を集約し、A/B の修正優先度付け。
- security-scan.yml (Semgrep/Trivy/gitLeaks) : コード/依存/シークレットの事前防御。
- cvp-readiness.md : これらを CVP (Defense/Red Team) の統制証拠として束ねる枠組み。

本ノートは防御・検知の観点に限定し、攻撃の再現手順や安全機構の回避方法は含めない。

運用レイヤー (SOC / 検知エンジニア向け)

上の観測点を、データソース → シグナル → 検知ロジックに落とす。ルール雛形はベンダ非依存の Sigma 風擬似記法 (防御用の検知であり、攻撃手順ではない)。閾値は各環境のベースラインに合わせて調整する。

1. データソース対応表

データソース	ここで効く段階	主なシグナル
フォワードプロキシ / NetFlow / DNS ログ	全段・最重要	LLM API・メッセージング API への外向き通信
コンテナランタイム監査 (Docker events / containerd / Falco / auditd)	侵入(Cairn)・ツール実行	特権 cap / host ネット / 未知イメージの起動
EDR / プロセステレメトリ (Sysmon / auditd / eBPF)	侵入・統括	スキャナ系プロセス、サブエージェント大量生成、連続自動ツール実行
Web/WAF アクセスログ	偵察(Strix)	パス列挙、多数 404/403、管理画面プローブ
ホスト永続化 (cron / systemd timer / 自動起動)	統括(Hermes)	新規スケジュール・サービス
認証 / 監査ログ	侵入・事後	異常な権限取得・横移動

2. 検知ルール雛形 (Sigma 風・擬似)

R1. サーバ系ホストから LLM API への外向き通信 (最優先)


```
logsource: proxy / dns / netflow
detection:
  dest_domain in:
    - api.openai.com
    - api.anthropic.com
    - api.openrouter.ai
    - api.deepseek.com
    - "*.openrouter.ai"
  src_zone: server|production|dmz # 開発者端末ゾーンは除外
condition: dest_domain AND src_zone
note: 正規業務でLLMを使うホストは allowlist 化し、それ以外からの通信を異常として上げる。
```

R2. 特権 cap / host ネットのコンテナ起動

```
logsource: container_runtime (docker events / falco / auditd)
detection:
  event: container_create|container_start
  any_of:
    - cap_add contains: NET_RAW
    - cap_add contains: NET_ADMIN
    - network_mode: host
    - privileged: true
condition: event AND any_of
note: cairn-worker-container 等、未知イメージ名との相関で確度を上げる。
```

R3. 偵察的 Web アクセス（列挙バースト）

```
logsource: web|waf
detection:
  status in: [401,403,404]
  uri in: ["/admin", "/administrator", "/wp-
login.php", "/phpmyadmin/", "/manager/html", "/console", ...]
  threshold: 同一 src_ip から 60秒に > N 件 (N は環境ベースライン)
condition: status AND (uri OR threshold)
note: 自動化ブラウザ(CDP/playwright)系UA/フィンガープリントと相関。
```

R4. サーバからのメッセージング API 通信 (C2 的チャネル)

```
logsource: proxy / dns
detection:
  dest_domain in:
    - api.telegram.org
    - discord.com / discordapp.com
    - slack.com / hooks.slack.com
    - *.signal.org
  src_zone: server|production
condition: dest_domain AND src_zone
note: サーバ/本番ホストからのメッセージング送信は強い異常シグナル。
```

R5. 永続化の新規作成

```
logsource: host (auditd / sysmon / file_events)
detection:
  any_of:
    - new cron entry (/etc/cron* , crontab -e)
    - new systemd timer/service (/etc/systemd/system/*.timer|.service)
    - new autostart item
condition: any_of
note: 自動化エージェントの常駐化。作成プロセス系統を併せて記録。
```

R6. 自律エージェントの相関検知（単発では見えないものを束ねる）

```
correlation (時系列・同一アセット or 同一オペレータ):
  R3 (偵察バースト) AND
  R1 (継続的な LLM egress) AND
  (R2 (特権コンテナ) OR R7 連続ツール実行) AND
  任意で R4/R5
window: 数十分～数時間
action: 「自律 AI エージェント攻撃」候補として優先度最上位でエスカレーション
note: 本キャンペーンの本質は“各単体は既知挙動でも、連鎖が速く広い”こと。相関が最大の武器。
```

R7. 1プロセス/エージェントが多数のセキュリティツールを短時間に連続実行

```
logsource: edr / process_telemetry (sysmon / auditd / ebpf)
detection:
  親プロセス（またはツール実行を仲介するプロセス）から、
  短時間に多種のセキュリティツールが子プロセスとして起動:
    nmap, ffuf, nikto, sqlmap, hydra, msfconsole, nuclei, gobuster, wpscan, ...
  threshold: 同一 parent から 5分に異なるツール >= N 種 (N は環境ベースライン)
condition: distinct_tool_count_by_parent >= N
note: ローカルLLM 利用で LLM egress(R1) が出ないエージェント (PentAGI/Villager 系) でも、
  この「人手を超える速度・多様さでのツール連続実行」は残る。MCP で 150+ ツールを
  束ねる基盤型 (HexStrike 系) にも効く。Kali ベースのコンテナ内実行なら R2 と相関。
```

3. MITRE ATT&CK 対応（検知の棚卸し用）

段階	代表 Technique	本ノートの検知
偵察	T1595 Active Scanning / T1046 Network Service Scanning	R3
実行・ツール	T1059 など（自動ツール実行）	EDR 連続実行、R2
C2	T1071 Application Layer Protocol（メッセージング悪用）	R4
永続化	T1053 Scheduled Task/Job（cron/timer）	R5
防御回避	T1562 Impair Defenses（安全機構の無効化に相当）	ガバナンス検知（※下記）
探索/横移動	T1046 / 認証異常	R2・認証ログ
持ち出し	T1041 Exfiltration over C2 Channel	R1/R4 と DLP の相関

※ T1562 相当（Hermes の「自身の安全フィルタ除去」など）は、ネットワーク/ホストの単純シグナルでは捉えにくい。LLM 利用ガバナンス（プロンプト/ツール呼び出しの監査、安全機構の除去・迂回の試行検知）で補う。本ノートは その「兆候の存在」を指摘するに留め、具体的な回避手法は扱わない。

3.1 MITRE ATLAS / OWASP-LLM への接続（補足）

主軸は ATT&CK（攻撃エージェントの“IT への行為”は古典的 TTP にそのまま乗るため、対応が強い）。一方 ATLAS と OWASP Top 10 for LLM は観点が異なるので、1:1 ではなく“接続点”として示す。正確さのため ID は公式マトリクスで要確認（atlas.mitre.org / genai.owasp.org）。

- MITRE ATLAS：主に「AI システムそのものへの/を使った敵対的脅威」。本件（AIが“攻撃者”として普通のITを攻める）とは向きが違うが、接続点はある。
 - 攻撃側が自分の LLM を兵器化する行為（安全機構の回避）＝ ATLAS の LLM Jailbreak / LLM Prompt Injection（例：AML.T0054 / AML.T0051、※要確認）→ 本ノートのガバナンス層（T1562 相当・G1-G6）で“兆候”を捉える。
 - 被害側が LLM 推論 API を露出（netcheck の 11434 等）＝攻撃面。外部からの推論悪用は ATLAS の ML Model Inference API Access 系に対応 → 露出は ③、不審な推論呼び出しは R1 の裏返し。
- OWASP Top 10 for LLM（2025）：主に「自組織の LLM アプリを守る」観点。接続点：
 - LLM06 Excessive Agency（自律エージェントが過大な権限で行動）＝本件の脅威モデルそのもの → 封じ込めは最小権限・ツール制限、検知は R7/R2・相関 R6。
 - LLM02 Sensitive Information Disclosure（情報持ち出し）→ R1/R4 と DLP の相関。
 - LLM01 Prompt Injection / LLM10 Unbounded Consumption（露出 LLM への注入・濫用）→ 露出は ③、濫用の兆候は R1。

まとめ：ATT&CK＝攻撃者の“行為”の共通語（強い対応）、ATLAS＝AI を巡る脅威の地図（接続点）、OWASP-LLM＝自組織 LLM アプリの守り（接続点）。新ルールには、まず ATT&CK ID、該当すれば ATLAS/OWASP-LLM ID を併記する。

4. SOC トリアージ手順（優先度順）

1. R1/R4 ヒット → 送信元ホストの業務用途を確認。正規でなければ即封じ込め（egress 遮断）＋プロセス/コンテナ調査。
2. R2 ヒット → 起動元・イメージ出所・cap の必要性を確認。未知イメージ＋特権は高確度。
3. R3 ヒット → 対象資産の露出（管理画面・RISKY ポート）を `netcheck` 等で確認し、塞ぐ。
4. R5 ヒット → 永続化を除去し、作成プロセス系統を遡る。
5. R6（相関） → 単発を超えた連鎖として、インシデント化・full IR へ。

5. 予防（検知の前段で効く固定策）

- ・ 送信ドメイン `allowlist`（サーバ系からの LLM/メッセージングは既定遮断）。
- ・ コンテナ：特権 `cap`・`host` ネット・未署名/未知イメージをポリシーで禁止（admission control）。
- ・ 攻撃面最小化：管理画面の隔離、DB/サービスをネット直結にしない（`netcheck` の RISKY ポート群）。
- ・ 決済・機微ページの CSP（skimmer/カード窃取コード注入の防止）。
- ・ LLM 利用の監査・ガバナンス（安全機構の除去/迂回の試行を検知・通報）。

5.1 管理面の既知 unauth-RCE（ベンダ製品の脆弱性）への備え

自律型の攻撃エージェントは、独自のゼロデイより「公開済みの既知 Critical を自動で突く」方が速い。特に認証不要で遠隔コード実行（unauth RCE）できる管理面は格好の標的になる。代表例：仮想基盤の管理面 vCenter（例：VMSA-2026-0006 / アドバイザリ ID 38017、最大 CVSS 9.8、認証回避→任意コード実行、回避策なし＝パッチのみ）。BMC/IPMI・各種管理 UI・アプライアンス管理面も同じ性質を持つ。これは「新しい AI の手口」ではなく 昔ながらの製品脆弱性だが、守りの型は本ノートと共通：

- ・ 予防(③)：管理面（vCenter/ESXi/VAMI 等）を管理用の隔離ネットワーク/VPN 内に限定し、ネット直結にしない。`netcheck` に管理面ポート（ESXi 902 / VAMI 5480 / vSphere UI 9443）を追加済み（露出の有無を確認）。
- ・ パッチ運用：ベンダのセキュリティアドバイザリ（VMSA / GHSA / ディストリ security 等）を購読し、認証不要・RCE・管理面に関わるものから最優先で適用。`vuln_triage` の CVE 分類もこの優先度で並べる。
- ・ 検知：管理面への偵察（R3）と認証異常を監視。侵入後は R2/R5/R7・相関 R6 に合流。
- ・ 注意（認可の境界）：露出確認・パッチ管理は自組織の資産に対してのみ。実際の脆弱性有無の“検証”は、自社の検証環境か、許可を得た範囲で。他者の運用基盤への実スキャンは利用者であることでは認可されない。

すべて検知・防御の設計に限定。攻撃の実行手順や安全機構の回避方法は本ノートに含めない。

6. 攻撃用 AI エージェントの広域マップと防御の当てはめ

Gambit の 3 ツール (Strix/Cairn/Hermes) は「分業チェーン型」の一例にすぎない。実際には AI ペンテスト/攻撃エージェントは多数あり、タイプで整理すると防御の当てはめが見通せる。朗報は、種類が増えても観測できる挙動の土台が共通 (LLM 推論の呼び出し・スキャン・ツール実行・任意で C2) であること。したがって R1-R7 の検知が横展開でほぼカバーでき、ツールごとに作り直す必要はない。

6.1 タイプ別マップ（防御観点）

類型	代表例	性質	主に効く検知
①分業チェーン型	Hermes+Strix+Cairn	役割を別ツールで分担（統括/偵察/侵入）	R1 R3 R2 R4 R5 R7+ 相関R6
④E2E 自律ペンテスト型	PentAGI, Villager, PentestAgent, Pentest-Swarm, hackingBuddyGPT 等	偵察～実行～検証を単体で完結（多くはマルチエージェント・Kali コンテナ）	R7 R2 R3（+ローカル LLMでなければ R1）
⑤攻撃基盤・フレームワーク型	HexStrike AI, CAI, TARS	エージェントに多数ツールを持たせる土台（MCP 等で 150+ ツール統括）	R7（多ツール連続実行）R2 R3
⑥対話型アシスタント型	PentestGPT, Pentest Copilot	人が主導し AI が提案/実行支援（自律モードを持つ物も）	R3 R7（自律時）／人手主導のため相対的に低速

類型名は一般的な整理の一例。個々の製品の分類や能力は変化するため、固定視せず挙動ベースで扱う。

6.2 新タイプで効きやすい検知の勘所

- ローカルLLM 対応型 (PentAGI/Ollama、Villager/DeepSeek ローカル等) は R1(LLM egress) を回避できる。 → R7（人手を超える速度・多様さのツール連続実行）とR2 (Kali 等コンテナ) に重心を移す。
- 基盤・フレームワーク型 (HexStrike 等) は MCP で大量ツールを束ねる。 → 1 プロセス/エージェント配下での多種ツール子プロセスの集中を R7 で捉える。
- E2E 型は Kali ベースのコンテナ内で実行することが多い。 → 未知/Kali 系イメージ+特権の起動を R2 で、内部の多ツール実行を R7 で、相関は R6 で。
- どの類型でも 攻撃面最小化・egress allowlist・特権コンテナ禁止（§5） が前段の固定策として効く。

6.3 悪用が報告/懸念されている例（注意喚起）

- **Villager** (Cyberspike, Kali+DeepSeek, PyPI で 1万超 DL) : 旧 AsyncRAT 配布インフラとの関連が指摘され、「次の Cobalt Strike」化が懸念。
- **HexStrike AI** (OSS, MCP で 150+ ツール統括) : Citrix の CVE-2025-7775 等を数分で悪用する用途が攻撃者フォーラムで共有されたと報告。
- **ARTEX** ([Autumn-27/ARTEX](#), AGPL-3.0, 百度BSRC「Agent+」優勝) : planner/worker+共有グラフ (Cairn に最も近い) +Hermes 型スキル+MCP のハイブリッド。記録型 MITM プロキシ・内蔵ガバナンス (承認ゲート/破壊的 deny/スコープ強制/LLM 記録) が特徴。韓国・金融機関の侵害での痕跡が報道 (未確定)。一次情報の静的解析と R1-R7/G1-G6 対応は [artex-analysis.ja.md](#) / [.en.md](#) 参照。

これらは「存在と危険性の注意喚起」。本ノートは検知・防御に限定し、各ツールの入手・実行・悪用手順は扱わない。

6.4 まとめ

攻撃側の“カタログ”は増え続けるが、**防御は挙動ベースで一般化できる**。本リポジトリの検知 ([detections/](#) の R1-R7 と相関 R6) と予防 (§5) は、Gambit の 3 ツールに限らず ④⑤⑥の多くに横展開できる。優先度は R1/R4 (egress) → R2 (特権コンテナ) → R7 (多ツール連続実行) → R3 (偵察) → R5 (永続化) → R6 (相関)。

参考: 攻撃用 AI エージェントの類型整理は 三輪信雄氏 (@NobMiwa) の公開まとめを参照した。個別ツールの出典は本ノート冒頭および §6.3 の記述に準ずる。

7. スキル機構とガバナンス (T1562 の押さえどころ)

ネットワーク (R1/R4)・ホスト (R2/R5/R7) の検知は「何が起きたか」を外側から観測する。これに対し、エージェント/LLM の“使い方そのもの”を統制・監査する層が必要になる領域がある。MITRE T1562 (Impair Defenses=防御の無効化) に当たる動きで、Hermes の「自身の安全フィルタ除去」のように、ネットワークやホストの単純なシグナルでは捕まえにくい。

7.1 スキルとは何か、なぜ兵器化の入口になるのか

エージェント基盤の「スキル」は、エージェントに後付けできる能力の束で、主に3要素から成る——指示 (プロンプト断片)、そのスキルが使ってよいツールの利用権、複数ステップのワークフロー。Hermes はさらに、作業経験からスキルを自動生成し、使いながら改良し、セッションをまたいで蓄積する自己改善ループを持つ。これ自体は正当で便利な機能である。

危険なのは、エージェントがスキルの内容を“信頼できる上位の指示+権限付与”として取り込む点にある。Gambit の事例では、攻撃者は Hermes 本体のコードを一行も変えず、攻撃用スキルを大量に (121 個中 78 個

が攻撃目的) 上乘せし、そのうち 1 つに「自身の安全フィルタを外す」役割を持たせただけで、 良性の汎用アシスタントを攻撃の司令塔に変えた。つまりスキルは、実質的に「信頼された指示+ツール権限を 外部から注入する口」であり、防御上はサプライチェーン（無審査のスキル取り込み）と プロンプトインジェクション（スキルの文章が判断を上書き）の二重の攻撃面になる。

構造的な理由だけ述べる：安全強制をスキル/モデル層の外側（スキルから触れない場所）に置かない設計だと、 スキルが「安全チェックを無視せよ」と指示する余地が生まれる。堅牢な設計は安全強制を外側に固定する。 本ノートは除去の具体手順は扱わない。

7.2 ガバナンスが具体的に要求していること

1. **不可変の安全層**：安全強制をエージェント内部ではなく、外側のゲートウェイ／プロバイダ側で行い、スキルやプロンプトから書き換えられないようにする。CVP の“不変の一線”（物理被害・大規模妨害・ランサム・安全系無効化は常にブロック）がこの実例。
2. **スキルの出所管理・allowlist**：署名済み・レビュー済みのスキルだけを許可し、出所を記録、 未知スキルの取り込みは既定で止める。社内レジストリで一元管理（＝サプライチェーン対策）。
3. **ツール権限の最小化+ブローカー経由**：エージェントに直接ツールを握らせず、ポリシーを強制する 仲介（ブローカー）越しに使わせる。権限拡大は人の承認を挟む。
4. **改ざん耐性のある監査ログ**：プロンプト・ツール呼び出し・スキルのロード/生成/自己改変・安全設定の変更を記録し、追記のみ／外部保管で改ざんに備える。ログの欠落・停止自体が異常シグナル。
5. **高リスク操作は人間承認（HITL）**：権限昇格・外部送信・破壊的操作はゲートで止める。
6. **検知との相関**：下記ガバナンス・シグナルを R1/R4/R7 等と時系列で突き合わせ（R6）、 “自律エージェントの始動”として束ねて見る。

7.3 監査イベント一覧（ガバナンス層・R1-R7 と同じ粒度）

これらは主に**アプリ/監査ログ層**（エージェントの監査ログ、スキルレジストリ、LLM/ツールのゲートウェイ、ログ基盤のヘルス）で観測する。ネットワーク/ホストの R1-R7 とは観測点が異なる。

ID	監査イベント / Signal	データソース	優先度	対応
G1	スキルの導入・自動生成・自己改変	エージェント監査ログ / スキルレジストリ	高	出所・署名・レビュー有無を確認。未承認なら隔離。
G2	安全設定・ガードレールの変更/無効化の試行	ポリシーゲートウェイ / エージェント設定ログ	高	外側の安全層で拒否。試行自体を即エスカレーション。
G3	プロンプト/ツール呼び出しに安全機構の迂回意図	LLM ゲートウェイ監査	高	ガバナンス審査。該当セッションを保全。
G4	監査ログの欠落・停止	ログ基盤ヘルス	高	ログ自体への攻撃を疑い、記録経路を復旧・保全。
G5	ツール権限の拡大要求/付与	ツールブローカー監査	中	最小権限に照合。拡大は人の承認を要求。
G6	サブエージェントの大量スポン／権限を広げる構成変更	プロセス/エージェント・テレメトリ	中	乱立・権限拡大のパターンを R7 と相関。

G1-G6 はネットワーク/ホストの Sigma/Falco/Datadog (detections/) ではなく、**ガバナンス/監査の層**で実装する。各組織のエージェント基盤・ログ基盤のスキーマに合わせて配線する。本ノートは兆候の存在と 対応方針のみを示し、安全機構の回避方法は一切含めない。

Cairn 動作確認ランブック（自分の隔離ラボ・認可済み前提）

`oritera/Cairn` (AGPL-3.0, v0.2.1) を、自分が所有・認可した、ネットワーク隔離された“やられ環境”に対して実際に動かすための設計書。

⚠ Cairn は能動的に脆弱性を突いてシェル/権限を奪う実行役。対象を所有しているか、書面で認可を得ている場合のみ実行すること。 本ランブックは `authorized pentest` / 自前ラボ / CTF を想定し、第三者・本番・未認可の対象には一切向けない。

i この Claude セッションの `sandbox` では実行しない (Docker デーモン無し・LLM キー無し・外部通信制限)。実行はあなたの隔離ラボ側で行う。

0. 実行前チェックリスト（すべて YES でなければ実行しない）

- ☐ 対象は**自分が所有**、または**書面の認可**がある（範囲・期間・対象 IP が明記）。
 - ☐ 対象と Cairn ワーカーは**独立したネットワークセグメント**に隔離（下記「ネットワーク隔離」）。
 - ☐ 対象・ワーカーとも**使い捨て or スナップショット済み**で、終了後に復元できる。
 - ☐ 外部通信は **LLM API エンドポイント（と初回のイメージ取得）**だけに限定。対象以外の内部ホスト・インターネットへ到達しない。
 - ☐ 予算・ステップ上限（タスク `timeout` / worker 数）を**保守的**に設定。
 - ☐ 実行中は**監視下**に置き、いつでも停止できる（`completed_action: stop` で事後調査可）。
-

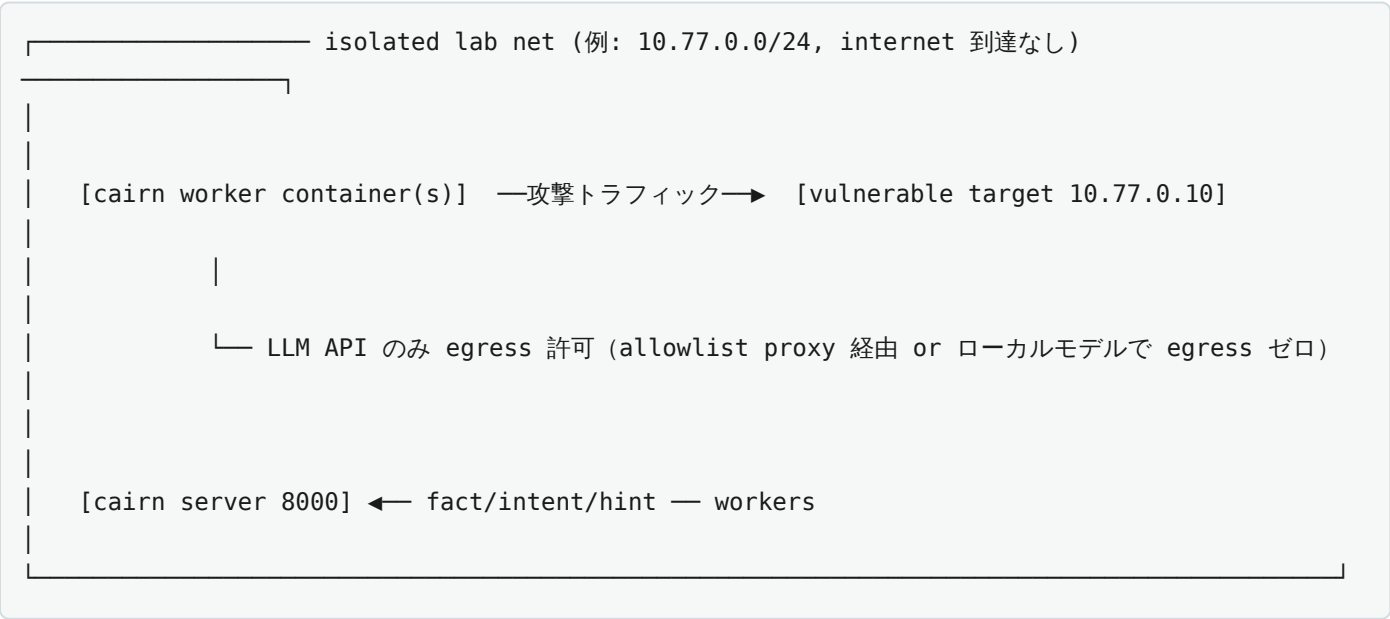
1. 構成要素

要素	役割	実行場所
Cairn Server (<code>cairn serve</code>)	黒板 (fact-intent グラフ) を持つ FastAPI。SQLite に状態保存。 <code>POST /projects</code> で <code>origin/goal</code> を登録	ラボ内のどこでも可 (UI/ API は 8000)
Dispatcher + Workers (<code>cairn dispatch</code>)	スケジューラ+OODA ワーカー。 <code>container</code> モード (<code>ghcr.io/oritera/cairn-worker-container:latest</code> を 1 プロジェクト 1 コンテナで <code>exec</code> 、Docker 必要) か <code>local</code> モード (ホストプロセス、 <code>claude/codex/pi</code> CLI を再利用)	隔離ネットワーク上
LLM プロバイダ	ワーカーの推論。 <code>worker env</code> で <code>ANTHROPIC_MODEL</code> / <code>BASE_URL</code> / <code>AUTH_TOKEN</code> (<code>claudecode</code> 型) 等	外部 API or ローカルモデル
やられ対象	<code>origin</code> に記載。例: <code>http://10.77.0.10 / goal: get a shell / capture the flag</code>	隔離ネットワーク上

ワーカーは `nmap -sS` など raw socket を使う場合 `cap_add: [NET_RAW, NET_ADMIN]` が要る。付けるのは必要時のみ。

2. ネットワーク隔離の設計（最重要）

`dispatch.example.yaml` は `network_mode: "host"` を例示しているが、隔離目的には不可（ホストのネットワーク全体に素通し）。代わりに：



- 専用の `internal` な Docker ネットワーク（または隔離 VLAN）を作り、対象とワーカーをそこへ。
- `egress` は LLM エンドポイントだけを許可（ファイアウォール allowlist）。可能ならローカルモデルにして `egress` を無くすのが最も安全。

- 対象以外の内部ホスト、インターネット、会社ネットワークが**無い**ことを実行前に確認（`ip route / 対象外 IP への ping` が全て失敗すること）。

3. 手順

3.1 ラボ準備

- 隔離ネットワークを作成（internet 到達なし）。
- やられ対象を立てる（自作の脆弱アプリ、または自分が用意した CTF 的ターゲット）。スナップショット取得。
- Cairn を動かすホスト（ワーカー）もスナップショット可能にする。

3.2 インストール

```
git clone https://github.com/oritera/Cairn && cd Cairn
uv sync --project cairn
# container モードなら worker イメージを取得（この時だけ egress 必要）
docker pull ghcr.io/oritera/cairn-worker-container:latest
```

3.3 設定（dispatch.yaml）

`dispatch.example.yaml` をコピーして編集。最低限：

- `server:` をラボ内のサーバ URL に。
- `workers[].env` に自分の LLM 資格情報（例 `ANTHROPIC_MODEL / ANTHROPIC_BASE_URL / ANTHROPIC_AUTH_TOKEN`）。
- `container.network_mode` は `host` をやめ、作成した隔離ネットワーク名を指定。`cap_add` は必要時のみ。
- `runtime.max_workers / tasks.*.timeout` を保守的に（まず小さく）。

秘密情報は `dispatch.yaml` にベタ書きせず環境変数や `secret` 管理から注入する。`dispatch.yaml` はリポジトリにコミットしない。

3.4 まず安全スモーク（mock・攻撃なし）

実標的に向ける前に、エンジンが回ることをモックで確認（§4 参照）。

3.5 本番実行

```
# 1) サーバ
uv run --project cairn cairn serve --host 0.0.0.0 --port 8000

# 2) プロジェクト作成 (origin=ラボ対象, goal=目標) : POST /projects
curl -s -X POST http://<server>:8000/projects \
  -H 'Content-Type: application/json' \
  -d '{"origin":"target http://10.77.0.10","goal":"get a
shell","bootstrap_enabled":true}'

# 3) ディスパッチャ起動 (ワーカーが OODA ループを回し始める)
uv run --project cairn cairn dispatch --config dispatch.yaml
```

3.6 観測・記録

- GET /projects/{id} で fact-intent グラフの成長を確認 (server の static UI でも可)。
- export.py ルータ (/export) で結果を取得し、ラボ記録として保存。
- completed_action: stop なのでワーカーコンテナを事後調査できる。

3.7 片付け

- ディスパッチャ/サーバ停止 → ワーカーコンテナ削除 → 対象とワーカーホストをスナップショットへ復元。
- egress allowlist を元に戻す。

4. 安全スモーク (mock) : 攻撃せずにエンジンだけ回す

type: mock ワーカーは LLM も実コマンドも使わず、確率的にダミーの Fact/Intent を出すだけ。

execution: local にすればコンテナも不要 (ホストの python3 サブプロセスで動く)。実標的に向ける前の配線確認に最適。

検証済み設定: dispatch_mock_local.yaml (dispatch_mock.yaml に runtime.execution: local + worker_healthcheck: disabled を足し、container: を除去したもの)。

```
# 依存は標準 PyPI から (Cairn の既定は aliyun ミラー固定なので上書き)
UV_DEFAULT_INDEX="https://pypi.org/simple" uv sync --project cairn

# 1) サーバ
UV_DEFAULT_INDEX="https://pypi.org/simple" uv run --project cairn cairn serve --host
127.0.0.1 --port 8000

# 2) 到達先の無いダミー project (title は必須)
curl -s -X POST http://127.0.0.1:8000/projects -H 'Content-Type: application/json' \
  -d '{"title":"mock-smoke","origin":"mock target (no network)","goal":"mock
goal","bootstrap_enabled":true}'

# 3) mock ディスパッチャ (ホストプロセス・攻撃なし)
UV_DEFAULT_INDEX="https://pypi.org/simple" uv run --project cairn cairn dispatch --
config dispatch_mock_local.yaml
```

- fact-intent グラフが OODA で成長していくのを `GET /projects/{id}` / static UI で観察。どこへもパケットは飛ばない。

この環境で検証済み (2026-10-06) : 上記手順で `cairn serve` 起動 → `proj_001` 作成 (origin / goal の 2 Fact) → mock ディスパッチャ起動で、OODA ループが `bootstrap→reason→explore` を回し、`f001-f003` のダミー Fact と `i001-i005` の Intent を生成。ディスパッチャログに外部 URL/接続は 0 件。Docker/LLM/標的すべて不要で完走した。本番の exploitation はラボ側のみ。

5. やってはいけないこと

- 未認可・第三者・本番環境を origin にする。
- `network_mode: host` のまま、隔離せずに実行する。
- egress を絞らず、対象以外へ到達できる状態で走らせる。
- 秘密情報を `dispatch.yaml` に残してコミットする。

この構成の実設定 (local / ローカルLLM / 隔離ネット)

選択した構成 (ローカルLLM・egress ゼロ / local 実行 / 既定の隔離ネット 10.77.0.0/24) の実ファイルを同梱 :

- `dispatch.lab.yaml` — そのまま使える local 実行設定。埋めるのは 2 値だけ : `workers[].env.CODEX_MODEL` (ローカルモデル名) と `CODEX_BASE_URL` (ローカルLLM の OpenAI 互換 URL) 。
- `run-lab.sh` — server 起動 → project 作成 (origin=認可ラボ対象) → dispatcher 起動 の手順ヘルパー。 `TARGET` / `GOAL` / `CAIRN_DIR` を環境変数で上書き可能。

local 実行はワーカーをホストプロセスで動かし、ホストの `codex CLI` を再利用する（サンドボックスなし）。したがって `dispatcher` ホスト自体を隔離セグメントに置くこと（`container.network_mode` ではなくホスト配置で隔離）。ローカルLLM のエンドポイントも隔離ネット内 or `localhost` に限定し、外部プロキシを設定しないことで egress ゼロを保つ。`codex CLI` がホストに導入済み・PATH 上にあることが前提。

参考（リポジトリ内）

- `dispatch.example.yaml` — container モードの完全な設定例
- `dispatch.local.example.yaml` — local モード（ホスト実行、CLI 再利用）
- `dispatch_mock.yaml` — mock ワーカー（攻撃なし）
- `docs/specs/server-protocol.md` — 黒板プロトコル / `POST /projects` (origin・goal)
- `docs/specs/dispatcher-design.md` — デイSPAッチャ設計

Strix をローカル実行する設計メモ（**Docker 不要の local** バックエンド）

対象: `usestrix/strix` v1.7.0 を、Docker デーモンを使わずに ホスト（例: Claude Code の cloud sandbox）上で直接動かす方法の調査メモ。

⚠ Strix は認可されたペネトレーションテスト用ツールです。対象は必ず自分が所有・許可を得たものに限定すること。

背景

Strix は「LLM API キーを入れれば動く」のではなく、スキャン対象を隔離した Docker コンテナを起動し、その中でエージェント（LLM ↔ ツール）を動かす設計。したがって本来の必須要件は 2 つ:

- LLM API キー（`STRIX_LLM` / `LLM_API_KEY`、LiteLLM の model id）
- 稼働中の Docker デーモン

cloud sandbox は Docker-in-Docker を通常持たないため（`docker info` が到達不可）、公式のローカル CLI モードはそのままでは動かない。マネージド版（`app.strix.ai`）は ネットワークポリシーで `app.strix.ai` への到達が拒否される場合がある。

実行フロー（コード地図）

```
strix/interface/main.py      エントリ (project.scripts: strix=...main:main)
├─ interface/cli.py  run_cli()   引数 → scan_config
│   └─ core/runner.py  run_strix_scan()
│       └─ runtime/session_manager.create_or_reuse() ← サンドボックス起動
│           └─ runtime/backends.get_backend(name) ← STRIX_RUNTIME_BACKEND で分岐
└─ エージェントループ (LLM ↔ tools/*)
```

- `runtime/backends.py`: `STRIX_RUNTIME_BACKEND`（既定 `docker`）でバックエンドを選ぶレジストリ。`register_backend(name, fn, supports_bind_mounts=...)` で独自バックエンドを追加できる。
- `runtime/docker_client.py`: `openai-agents` SDK の `DockerSandboxClient` を継承し、コンテナに `NET_ADMIN`/`NET_RAW`（`nmap -sS` 等の raw socket 用）、`host.docker.internal` ゲートウェイを付与。
- `tools/shell` の `exec_command` は SDK 提供（`agents.sandbox.capabilities.tools.shell_tool.ShellTool`）。エージェントの全コマンド実行はサンドボックスセッション経由 → 「セッション = 実行環境」。

結論：ゼロから書く必要はない

openai-agents SDK に `UnixLocalSandboxClient` (ローカル Unix サンドボックス) が同梱されている。

```
.venv/.../agents/sandbox/sandboxes/unix_local.py
class UnixLocalSandboxClient      # create() で session を返す
class UnixLocalSandboxSession    # ホスト上で直接コマンド実行、workspace = manifest.root (ホスト FS)
```

Docker 版 `_docker_backend` の実体は「SDK の Client を作って session を返す」だけなので、これを Unix 版へ差し替えるだけで成立する。

実装案 (新バックエンド ~15 行)

`strix/runtime/backends.py` に追加:

```
async def _unix_local_backend(
    *, image, manifest, exposed_ports, bind_mounts=None,
):
    # image と bind_mounts は使わない (ホスト FS がそのまま workspace)
    from agents.sandbox.sandboxes.unix_local import (
        UnixLocalSandboxClient, UnixLocalSandboxClientOptions,
    )
    client = UnixLocalSandboxClient()
    options = UnixLocalSandboxClientOptions(exposed_ports=exposed_ports)
    session = await client.create(options=options, manifest=manifest)
    await session.start()
    return client, session

# bind mount 非対応 (= manifest entries 経由でソースを materialize)
register_backend("local", _unix_local_backend, supports_bind_mounts=False)
```

起動:

```
export STRIX_RUNTIME_BACKEND=local
export STRIX_LLM="anthropic/claude-sonnet-5-5"  # LiteLLM の model id
export LLM_API_KEY=...
uv run strix -n -t ./ --scan-mode quick --max-budget 10
```

`get_backend()` が `STRIX_RUNTIME_BACKEND` で分岐するため、既存の Docker 経路には触れない。

注意点（ここが実質的な作業）

#	論点	内容
①	ツールの非同梱	nmap / ffuf / caido(プロキシ) / playwright 等は Docker イメージ側に同梱。ローカルには無い → 使うツールだけホストに apt / pip で導入が必要。
②	manifest の users/groups 不可	UnixLocalSandboxSession は manifest の user/group 指定を非サポート（コードに明示）。session_manager._host_identity_env が uid/gid を積むため、local 経路では渡さない調整が要る。
③	caido 前提の除外	Docker 版は起動時に bootstrap_caido() で in-container プロキシ(48080)へ接続。ローカルには無い → caido をホスト起動するか、proxy ツール依存フローを無効化。
④	分離性の喪失（最重要）	Docker 隔離をやめてホストで直接ツール実行 = 攻撃実行環境がホストになる。対象は自分が権限を持つものに限定。nmap -sS 等は root/cap が必要。
⑤	bind mount vs materialize	supports_bind_mounts=False なら session_manager.build_manifest_entries() が LocalDir でソースを workspace に展開。-t ./ のローカルコード解析はこれで動く。

推奨スコープ（最小 PoC）

-t ./（ローカルコードの静的解析 = code_review 相当）に限定すれば、ネットワーク系ツール（nmap/caido）が不要になり、①③④をほぼ回避できる。まずこの構成で PoC するのが現実的。実スキャンには LLM API キーが必要。

確認済みの事実（この環境での動作確認）

- uv sync で依存導入、uv run strix --help 正常 (v1.7.0)。
- docker info 到達不可（ローカル CLI の実スキャンは不可）。
- strix cloud login はネットワークポリシーにより app.strix.ai へ 403（プロキシ CONNECT 拒否）。
- SDK に unix_local.py が存在することを確認。

PoC 結果（実装して検証した）

設計案どおり 2 ファイルにパッチを当て、PoC を実行して成立を確認した。パッチ: `strix-local-poc/strix-local-backend.patch` 検証スクリプト: `strix-local-poc/poc_local_backend.py`

当てたパッチ (3 ファイル)

- strix/runtime/backends.py: `_unix_local_backend` を追加し `_BACKENDS["local"]` に登録。さらに `caido` 対応をバックエンド単位で持つレジストリ `_CAIDO_BACKENDS`（既定 `{"docker"}`）と `backend_supports_caido()` を追加。 `register_backend` に `supports_caido` 引数を追加。
- strix/interface/main.py: Docker 事前チェック（`check_docker_installed` / `pull_docker_image`）を `settings.runtime.backend == "docker"` のときだけ実行するようゲート。

3. `strix/runtime/session_manager.py`: `create_or_reuse` の `caido` 結合をバックエンド単位でゲート。 `caido` 非対応バックエンド (`local`) では (a) `caido` プロキシ env (`http_proxy` / `https_proxy` / `ALL_PROXY`) を注入しない、(b) `exposed_ports=()`、(c) `resolve_exposed_port` と `bootstrap_caido` をスキップ、(d) `caido_client=None`。 proxy ツールは既存の `graceful-degradation` (`_ctx_client` が `None` → 「利用不可」) に乗る。

検証 1: バックエンドを直接叩く (LLM 非依存)

`poc_local_backend.py` を `uv run python` で実行した結果:

- `supported_backends()` → `['docker', 'local']`、`get_backend('local')` 解決 OK。
- `DOCKER_HOST=unix:///nonexistent-docker.sock` にしても **セッション起動成功** (`UnixLocalSandboxSession`)。 Docker デーモンに一切触れない。
- `session.exec(...)` が **ホスト上で実行される** (`uname -a` が実ホスト、 `whoami=root`、 `python3 --version=3.13.16`)。
- `LocalDir` で対象ソースが `workspace` に `materialize` され、 `ls repo` に `netcheck` / `scripts` / `security-scan` / ... が出現。
- 対象リポジトリ自身のツール `scripts/summarize.py` をセッション内で実行し完走 (`exit 0`)。

検証 2: 実 CLI 配線 (`strix -n -t ./ --scan-mode quick`)

`STRIX_RUNTIME_BACKEND=local` で CLI を起動:

- パッチ前: 起動時の Docker 事前チェックで `DOCKER NOT AVAILABLE` により即停止。
- パッチ後: Docker ゲートを通過し、環境検証 → **LLM プリフライト接続**まで到達。 ダミーキーのため `invalid x-api-key` で停止 (`api.anthropic.com` への到達自体は成功)。 → **残る唯一の前提は有効な LLM API キーのみ**。 実キーがあれば検証 1 で実証済みの ローカルセッション起動を経てエージェントループに進む。

検証 3: caido 非依存 (`create_or_reuse` を `local` で直接呼ぶ)

`poc_caido_free.py` を実行した結果:

- `backend_supports_caido('local')=False` / `('docker')=True`。
- `create_or_reuse(backend=local)` が **`caido bootstrap` を呼ばずにセッションを返す**。
- `bundle["caido_client"] is None` (proxy ツールは「利用不可」に `degrade`。クラッシュしない)。
- セッション内に `caido` プロキシ env が **注入されていない** (`echo $http_proxy $ALL_PROXY` が空)。
- 対象ソースの `materialize` と `session.exec` は正常。

→ これで `local` バックエンドは **`caido` サイドカー無しで完結**する。コード解析系スキャン (`proxy` ツールを使わないフロー) なら、追加の外部依存なしに動かせる。

PoC で判明した追加の実挙動 (設計メモの補足)

- `base_dir = Path.cwd()` 制約: SDK の `LocalDir` 展開は、ソースが CWD 配下でないと `LocalDirReadError (outside_base_dir)` で拒否する (`source grant` 未指定時)。 `Strix` の正規の使い方 `-t ./` (対象=カレント) では常に満たされるため実害なし。 CWD 外を対象にする場合は `source grant` の付与か、対象を CWD 配下に置く必要がある。

- **Docker 事前チェックの無条件実行:** `main.py` の `check_docker_installed` / `pull_docker_image` はバックデンド設定を見ずに走るため、`local` でも上記ゲートが必須。
- シンボリックリンクは `LocalDir` 展開で非サポート (`symlink_not_supported`)。 `.git` などリンクを含むツリーは除外して渡すのが無難。

未検証（本 PoC のスコープ外）

- 実 LLM キーでのエンドツーエンドのスキャン完走。
- 動的スキャン (`live_test`) で `proxy` ツールを使うフロー。 `local` では `caido` が無いため トラフィック捕捉系は使えない（コード解析系に限定すれば問題なし）。
- `nmap` など外部ツールを要する動的スキャン（ホストに未導入）。